

Avaliação Comparativa de Políticas de Escalonamento de Processos e Proposição do Algoritmo Triagem com Espera Justa (TEJ)

André Wesley Barbosa Rodrigues Filho
Engenharia de Software
Universidade Federal do Cariri (UFCA)
Juazeiro do Norte, Brasil
andre.wesley@aluno.ufca.edu.br

Leôncio Ferreira Flores Neto
Engenharia de Software
Universidade Federal do Cariri (UFCA)
Juazeiro do Norte, Brasil
leoncio.ferreira@aluno.ufca.edu.br

Paulo Gabriel Leite Landim
Engenharia de Software
Universidade Federal do Cariri (UFCA)
Juazeiro do Norte, Brasil
landim.gabriel@aluno.ufca.edu.br

Salomão Rodrigues Silva
Engenharia de Software
Universidade Federal do Cariri (UFCA)
Juazeiro do Norte, Brasil
salomao.silva@aluno.ufca.edu.br

Abstract—O escalonamento de processos é uma parte essencial dos sistemas operacionais modernos, sendo responsável por dividir o uso da CPU entre vários processos com comportamentos diferentes. Algoritmos baseados em prioridade favorecem tarefas importantes, mas podem fazer com que processos de menor prioridade nunca consigam rodar (problema conhecido como inanição ou *starvation*) quando a carga do sistema está muito alta. Este artigo propõe um novo algoritmo de escalonamento, a Triagem com Espera Justa (TEJ), e avalia seu desempenho em comparação com três algoritmos clássicos (FCFS, Prioridade Estática Não Preemptiva e Round-Robin) por meio de simulação. Inspirado na triagem de pronto-socorro de hospitais, o TEJ combina prioridade com um tempo máximo de espera na fila de prontos, dando prioridade de atendimento para os processos resgatados. O TEJ faz isso de forma simples, sem precisar interromper processos no meio da execução (não preemptivo) e sem tentar adivinhar o tempo de execução futuro das tarefas. Os testes experimentais envolveram 1.600 simulações usando 100 sementes de aleatoriedade diferentes, divididos em quatro cenários de teste com 1.000 processos cada. Os resultados mostram que, em cenários com muitas tarefas de alta prioridade, o TEJ aumentou o índice de justiça de Jain (aplicado ao *slowdown* de cada processo) de 69,93% para 88,61% em comparação com a Prioridade Pura. Além disso, o TEJ manteve o mesmo número reduzido de trocas de contexto dos algoritmos não preemptivos e superou o Round-Robin em velocidade de execução.

Index Terms—Sistemas Operacionais, Escalonamento de Processos, Inanição (*Starvation*), Triagem com Espera Justa, Simulação por Eventos Discretos, Índice de Justiça de Jain.

I. INTRODUÇÃO

O gerenciamento e o escalonamento da CPU são partes fundamentais de qualquer sistema operacional moderno [1], [2]. O papel do escalonador é decidir a ordem em que os processos vão rodar na CPU, tentando equilibrar objetivos que muitas vezes são conflitantes: aumentar a quantidade de tarefas finalizadas (*throughput*), reduzir o tempo total que um processo leva para rodar (*turnaround time*), diminuir os custos de troca

de contexto e garantir que todas as tarefas sejam tratadas de forma justa (*fairness*) [3].

Para entender os desafios de organizar os processos quando há muitas tarefas no sistema, imagine a **triagem em um pronto-socorro de hospital** (como o Protocolo de Manchester). Na recepção, os pacientes são divididos por gravidade (prioridade): casos graves (pulseira vermelha) devem ser atendidos na hora pelo médico (CPU), enquanto casos leves (pulseira verde) esperam na sala (fila de prontos). Porém, se pacientes graves não pararem de chegar, as pessoas com casos leves nunca serão atendidas, sofrendo **inanição médica** (*starvation*). Por outro lado, interromper uma consulta médica no meio para atender outro paciente gera perda de tempo (o médico precisa ler a ficha do novo paciente e entender o caso), o que equivale ao custo de **troca de contexto** (δ) nos computadores.

Na computação, abordagens clássicas tratam esse dilema de formas extremas. Políticas focadas em tempo compartilhado, como o *Round-Robin* (RR), promovem justiça ao custo de inúmeras interrupções. Políticas focadas em precedência, como a *Prioridade Estática*, evitam interrupções, mas condenam processos menos importantes à inanição perpétua sob alta carga [4]. Adicionalmente, técnicas tradicionais contra inanição como o envelhecimento (*aging*) [1] utilizam escalonamento dinâmico que compromete a previsibilidade do prazo máximo de espera.

Neste trabalho, propõe-se o algoritmo **Triagem com Espera Justa (TEJ)** como uma solução para esses problemas. Baseado na triagem de hospitais, o TEJ mantém o atendimento prioritário por gravidade, mas define para cada processo um limite de espera na fila. Se o processo atingir esse limite, recebe precedência na próxima decisão do escalonador. Isso estabelece um limite determinístico para a espera na fila de prontos, sem interromper processos em andamento e sem tentar adivinhar a duração futura das tarefas.

O restante deste artigo detalha o modelo do sistema (Seção

II), a lógica matemática do TEJ e o funcionamento dos outros algoritmos (Seção III), o formato dos experimentos e testes (Seção IV), os resultados detalhados com gráficos (Seção V), a discussão das vantagens e desvantagens do TEJ (Seção VI) e a conclusão do trabalho (Seção VII).

II. MODELO DO SISTEMA E SIMULAÇÃO

A. Representação e Estados do Processo

O simulador opera no modelo de tempo discreto orientado a *ticks*. Cada processo P_i é modelado como uma tupla contendo seu identificador unívoco ID , tempo de chegada à simulação (T_{arr}), prioridade estática $Pr \in [0, 10]$ e uma sequência finita alternada de rajadas de computação e operações de entrada/saída (E/S):

$$P_i = \langle CPU_1, ES_1, CPU_2, ES_2, \dots, CPU_k \rangle \quad (1)$$

Por convenção adotada uniformemente em todo o projeto, valores numéricos menores indicam prioridade mais alta (0 representa a prioridade máxima e 10 a prioridade mínima). O ciclo de vida do processo percorre cinco estados discretos:

- 1) **Novo (NEW)**: O processo foi instanciado pela carga sintética e aguarda o instante $t = T_{arr}$ para ingressar no sistema.
- 2) **Pronto (READY)**: O processo encontra-se na fila de aptos aguardando a seleção pelo escalonador. O instante de sua inserção mais recente é registrado em T_{ready_arr} .
- 3) **Executando (RUNNING)**: O processo aloca com exclusividade a CPU.
- 4) **Bloqueado (BLOCKED)**: O processo concluiu uma rajada de CPU e aguarda a conclusão assíncrona de sua operação de E/S.
- 5) **Finalizado (TERMINATED)**: O processo concluiu sua última rajada de CPU e registrou seu instante final de término T_{fin} .

B. Modelagem de E/S Paralela e Retorno à Fila

As operações de E/S são modeladas como canais de dispositivos independentes e paralelos. Múltiplos processos podem permanecer no estado bloqueado simultaneamente sem sofrer contenção mútua de barramento. A cada *tick*, o tempo restante de E/S de cada processo bloqueado é decrementado unitariamente.

Quando um processo conclui sua requisição de E/S, ele transita imediatamente para o estado **READY** e é enfileirado na cauda da fila de prontos. O instante dessa reinserção atualiza $T_{ready_arr} = t_{atual}$, iniciando um novo ciclo de espera limpo para efeitos de cálculo de inanição. A mesma modelagem de E/S foi aplicada a todos os algoritmos em todos os experimentos, garantindo isolamento experimental e estrita reprodutibilidade.

C. Custo de Troca de Contexto e Indisponibilidade de CPU

A alternância do processo em execução na CPU consome sobrecarga de hardware e software (salvamento e restauração de registradores e tabelas de páginas) [4]. No simulador de eventos discretos utilizado para a validação, o custo de troca

de contexto (δ) é configurável e foi fixado em $\delta = 1$ *tick* para todos os experimentos principais.

As seguintes regras determinísticas governam a troca: ela ocorre sempre que um processo deixa a CPU (por E/S, término ou preempção) e outro diferente é despachado. Durante o intervalo $[t, t + \delta)$, a CPU entra no estado *SWITCHING* e fica indisponível. A troca não é cobrada ao sair do estado ocioso (*IDLE*).

D. Geração Determinística de Cargas e Sementes

A geração de processos emprega um Gerador Congruencial Linear (LCG) com sementes pseudoaleatórias de 32 bits. Os tempos de chegada seguem rigorosamente uma distribuição uniforme discreta no intervalo $[0, 5000]$ *ticks* em todos os cenários, garantindo que a pressão de carga na fila de prontos seja idêntica entre os algoritmos para uma mesma semente.

III. ALGORITMOS DE ESCALONAMENTO

A. Algoritmos Clássicos de Referência (Visão Geral)

- **FCFS (First-Come, First-Served)**: Política não preemptiva orientada à ordem de chegada (T_{ready_arr}); desempate por menor ID .
- **Prioridade Estática Não Preemptiva**: Seleciona o menor $Pr \in [0, 10]$ (0 = máxima). Desempate por menor T_{ready_arr} e menor ID . Executa sem preempção.
- **Round-Robin (RR)**: Preemptivo com quantum $q = 10$ ticks. Desempate por menor T_{ready_arr} e menor ID . Custo de troca: $\delta = 1$ *tick*.

B. Algoritmo Proposto: Triagem com Espera Justa (TEJ)

A descrição detalhada a seguir é dedicada ao algoritmo próprio concebido pela equipe, contemplando sua motivação, funcionamento, informações utilizadas, decisões de projeto e limitações.

1) *Motivação e Problema a Resolver*: O TEJ foi projetado especificamente para solucionar a **inanição (starvation)** de processos de menor prioridade submetidos a fluxos contínuos de alta prioridade. A concepção utiliza a metáfora da triagem de pronto-socorro: pacientes graves devem ser atendidos primeiro, porém o acúmulo de emergências não pode deixar pacientes comuns esperando infinitamente na recepção.

2) *Informações Utilizadas (Causalidade Estrita)*: O TEJ opera exclusivamente com informações de passado e presente (causalidade estrita): prioridade estática original ($Pr_i \in [0, 10]$), instante de entrada mais recente na fila de prontos ($T_{ready_arr,i}$), instante corrente do relógio (t), e o parâmetro de intervalo de resgate ($rescue_interval = 10$).

3) *Funcionamento e Fórmula do Prazo de Resgate*: Ao ingressar na fila de prontos, determina-se o limite máximo de espera (ΔW_i):

$$\Delta W_i = rescue_interval \times (Pr_i - Pr_{min} + 1) \quad (2)$$

onde $Pr_{min} = 0$. O prazo absoluto de resgate é $T_{deadline,i} = T_{ready_arr,i} + \Delta W_i$. O processo é considerado **resgatado** quando $t \geq T_{deadline,i}$.

Quando a CPU atinge o estado livre, a seleção ocorre em duas fases:

- 1) **Fase de Resgate (Precedência Absoluta):** Se houver processos resgatados, escolhe-se o de menor $T_{deadline}$ (desempate por menor T_{ready_arr} e menor ID).
- 2) **Fase Normal de Prioridade:** Caso contrário, escolhe-se o de maior prioridade estática (menor Pr , desempate por menor T_{ready_arr} e menor ID).

4) *Análise de Escolhas de Projeto e Trade-Offs:* A concepção do TEJ envolveu escolhas arquiteturais deliberadas para priorizar a prevenção da inanição sem comprometer a eficiência computacional. A Tabela I resume os principais *trade-offs* analisados no projeto.

5) *Limitações Teóricas do TEJ:* A principal limitação decorre da não preempção: rajadas longas de CPU em andamento atrasam o início do atendimento de emergências recém-chegadas. Além disso, a concessão de precedência absoluta a tarefas resgatadas eleva marginalmente o turnaround médio global em relação à Prioridade Pura.

IV. METODOLOGIA EXPERIMENTAL

A. Cenários Obrigatórios Avaliados

Foram configurados quatro cenários experimentais para submeter os algoritmos a regimes operacionais contrastantes:

- 1) **Equilibrado (*balanced*):** Carga mista com probabilidade uniforme de rajadas de CPU curtas e longas, taxas balanceadas de E/S e 50% de processos de alta prioridade.
- 2) **I/O-Bound (*io_bound*):** Processos com rajadas curtas de CPU ($[1, 5]$ ticks) e alta frequência de requisições longas de E/S ($[30, 100]$ ticks), conforme `configs/io_bound.conf`.
- 3) **CPU-Bound (*cpu_bound*):** Processos com rajadas longas de processamento ($[50, 200]$ ticks) e baixa incidência de E/S.
- 4) **Prioridades Desbalanceadas (*unbalanced_priorities*):** Carga altamente estressante onde 85% dos processos nascem com prioridade 0 (máxima) e 15% nascem com prioridade 10 (mínima), conforme `configs/unbalanced_priorities.conf`, criando o ambiente clássico indutor de inanição.

B. Métricas de Avaliação e Formulação

1) *Tempo Médio de Retorno (Average Turnaround):* Mede o intervalo transcorrido entre a chegada do processo ao sistema e sua conclusão definitiva:

$$\bar{T}_{\text{turnaround}} = \frac{1}{N} \sum_{i=1}^N (T_{\text{fin},i} - T_{\text{arr},i}) \quad (3)$$

2) *Trocas de Contexto (Context Switches):* Contabiliza o número total de eventos de substituição do processo ativo na CPU, mensurando a sobrecarga imposta pelo escalonador ao sistema.

3) *Slowdown e Índice de Justiça de Jain:* O *slowdown* individual (S_i) expressa a razão entre o tempo de permanência real no sistema e o tempo mínimo ideal de execução de CPU, desconsiderando a espera e as operações de E/S na linha de base:

$$S_i = \frac{T_{\text{fin},i} - T_{\text{arr},i}}{\sum \text{CPU}_i} \quad (4)$$

A justiça no tratamento dos processos é quantificada pelo Índice de Jain [5] aplicado aos valores de *slowdown*:

$$J(S) = \frac{\left(\sum_{i=1}^N S_i\right)^2}{N \sum_{i=1}^N S_i^2} \times 100\% \quad (5)$$

Valores próximos a 100% indicam que todos os processos sofreram penalização proporcional equivalente, enquanto valores reduzidos apontam disparidade severa e inanição.

C. Ambiente de Execução e Especificações de Hardware

Para garantir a integridade da medição e eliminar variabilidades externas de ambiente, todas as 1.600 simulações da campanha experimental foram executadas de forma dedicada em um ambiente de hardware controlado com as seguintes especificações:

- **Modelo da Maquinaria:** Notebook Acer Nitro ANV15-51;
- **Processador:** Intel Core i7 (Intel64 Family 6 Model 186 Stepping 2 @ 2,40 GHz);
- **Memória RAM:** 16 GB (16.088 MB físicos);
- **Sistema Operacional:** Microsoft Windows 11 Home Single Language (64-bit, Compilação 26200);
- **Ferramental de Compilação:** Compilador GCC com sinalizadores de otimização (`-Wall`, `-Wextra`, `-pthread`) e Python 3 para automação estatística e geração dos gráficos vetoriais.

D. Campanha Experimental e Reprodutibilidade

A campanha completa totalizou **1.600 execuções** (4 algoritmos $\times$ 4 cenários $\times$ 100 sementes). Cada simulação processou individualmente **1.000 processos**, contabilizando 1,6 milhão de processos analisados. Para garantir 100% de reprodutibilidade, as 100 sementes numéricas foram catalogadas em `configs/seeds.txt` e a automação das simulações foi orquestrada via regras de *Makefile* (`make run-all`).

Para cada métrica, calculou-se a média amostral ($\bar{X}$), o desvio padrão (s) e o Intervalo de Confiança de 95% (IC 95%) com base na distribuição t de Student ($n = 100$):

$$\text{IC}_{95\%} = \bar{X} \pm t_{0.025,99} \cdot \frac{s}{\sqrt{n}} \approx \bar{X} \pm 1,9842 \cdot \frac{s}{\sqrt{100}} \quad (6)$$

V. RESULTADOS E ANÁLISE QUANTITATIVA

Os resultados consolidados da campanha experimental estão detalhados na Tabela II e visualizados nas Figuras 1, 2 e 3.

Tabela I
ANÁLISE DE ESCOLHAS DE PROJETO E TRADE-OFFS DO ALGORITMO TEJ

Decisão de Projeto	Escolha Adotada	Trade-Off
Política Não Preemptiva	Execução contínua até o fim da rajada corrente.	Reduz trocas de contexto drasticamente; nova emergência aguarda fim da rajada atual.
Fila de Resgate vs. Aging Tradicional	Precedência direta após prazo rígido $T_{deadline}$.	Garantia determinística de espera máxima (teto de 110 ticks), sem ajustes fluidos de prioridade.
Precedência Absoluta do Resgate	Processos resgatados prevalecem sobre os não resgatados na seleção.	Reduz a espera indefinida; aceita pequeno acréscimo no turnaround de processos prioritários.
Reinício de Espera pós-E/S	Contagem de espera zerada a cada retorno de E/S.	Avalia inanição apenas no período em que o processo está apto a usar CPU.

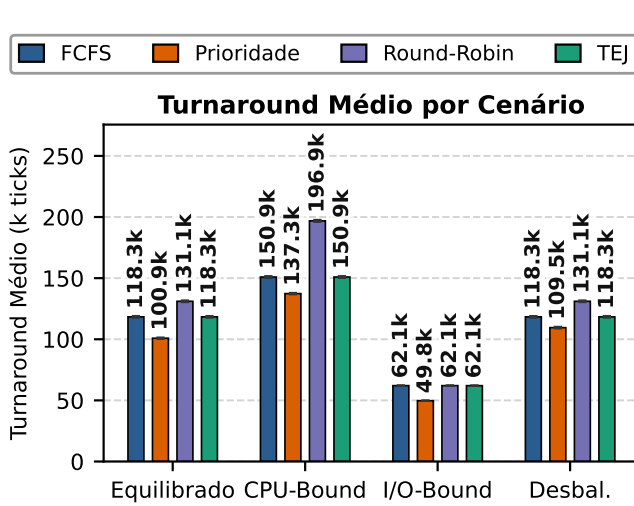


Fig. 1. Comparação do Tempo Médio de Retorno (*Average Turnaround*) com barras de erro indicando IC 95% nos quatro cenários avaliados.

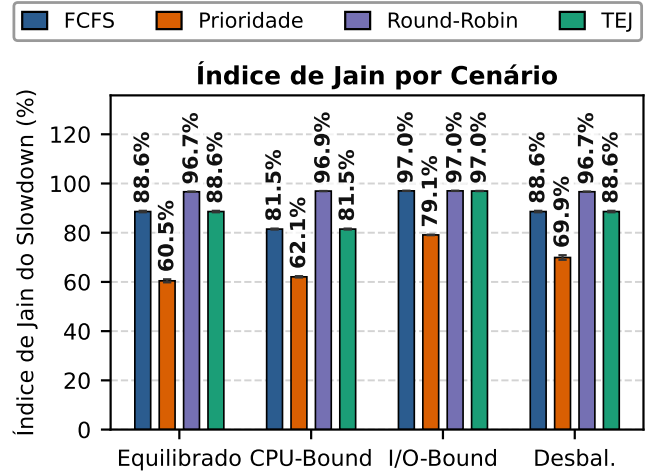


Fig. 3. Índice de Justiça de Jain (%) aplicado ao *slowdown*. O TEJ recupera o índice de justiça em relação à Prioridade Clássica no cenário desbalanceado.

A. Análise do Tempo Médio de Retorno

Conforme ilustrado na Figura 1, o escalonador por Prioridade Estática obteve o menor tempo médio de retorno global em todos os quatro cenários (100.856,68 no Equilibrado; 49.757,02 no I/O-Bound; 137.349,30 no CPU-Bound). Esse comportamento decorre de sua política de despacho: ao sempre executar os processos de maior prioridade primeiro, ela antecipa tarefas que, por serem priorizadas, tendem a completar antes, reduzindo o acúmulo de tempo na fila de prontos.

O algoritmo TEJ apresentou tempo médio de retorno muito próximo ao FCFS (118.341,93 vs. 118.338,47 no Equilibrado; 118.337,73 no Desbalanceado), com intervalos de confiança amplamente sobrepostos, indicando resultados muito semelhantes para o turnaround médio global enquanto garante o resgate ordenado dos processos.

O Round-Robin apresentou o pior tempo de retorno em todos os cenários (196.865,69 no CPU-Bound), penalizado pela fragmentação de rajadas longas e pelo acúmulo contínuo de custos de troca de contexto ($\delta = 1$).

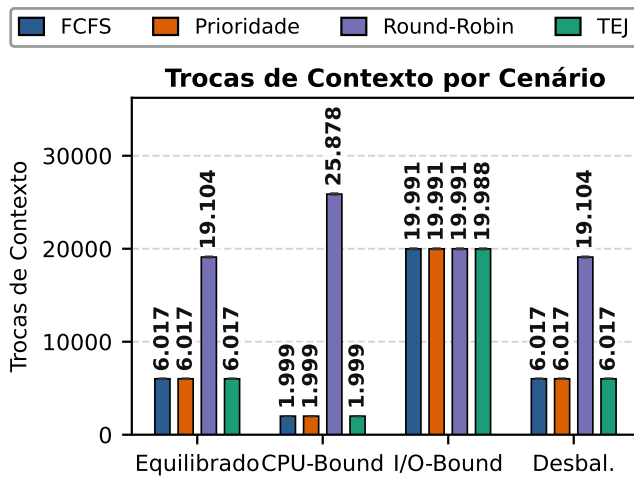


Fig. 2. Quantidade total de Trocas de Contexto nos quatro cenários (escala com IC 95%). Destaca-se a sobrecarga excessiva introduzida pelo Round-Robin.

Tabela II
CONSOLIDAÇÃO DOS RESULTADOS EXPERIMENTAIS COM MÉDIA AMOSTRAL E INTERVALO DE CONFIANÇA DE 95% ($N = 100$ SEMENTES POR CENÁRIO, 1.000 PROCESSOS POR EXECUÇÃO)

Cenário	Algoritmo	Turnaround Médio (ticks)	Trocas de Contexto	Jain do Slowdown (%)
Equilibrado (<i>balanced</i>)	FCFS	118.338,47 $\pm$ 630,48	6.016,84 $\pm$ 14,43	88,61% $\pm$ 0,29%
	Prioridade	100.856,68 $\pm$ 527,71	6.016,84 $\pm$ 14,43	60,45% $\pm$ 0,62%
	Round-Robin ($q = 10$)	131.071,78 $\pm$ 650,77	19.103,97 $\pm$ 48,01	96,65% $\pm$ 0,04%
	TEJ (Proposto)	118.341,93 $\pm$ 630,96	6.016,84 $\pm$ 14,43	88,60% $\pm$ 0,29%
CPU-Bound	FCFS	150.921,23 $\pm$ 687,79	1.999,49 $\pm$ 5,09	81,48% $\pm$ 0,20%
	Prioridade	137.349,30 $\pm$ 540,79	1.999,49 $\pm$ 5,09	62,08% $\pm$ 0,27%
	Round-Robin ($q = 10$)	196.865,69 $\pm$ 739,41	25.878,36 $\pm$ 69,97	96,93% $\pm$ 0,03%
	TEJ (Proposto)	150.924,82 $\pm$ 687,22	1.999,49 $\pm$ 5,09	81,48% $\pm$ 0,20%
I/O-Bound	FCFS	62.077,49 $\pm$ 179,98	19.990,79 $\pm$ 42,16	97,01% $\pm$ 0,03%
	Prioridade	49.757,02 $\pm$ 136,12	19.990,67 $\pm$ 42,12	79,11% $\pm$ 0,11%
	Round-Robin ($q = 10$)	62.077,49 $\pm$ 179,98	19.990,79 $\pm$ 42,16	97,01% $\pm$ 0,03%
	TEJ (Proposto)	62.050,54 $\pm$ 179,76	19.988,32 $\pm$ 42,11	96,96% $\pm$ 0,03%
Prioridades Desbalanceadas (<i>unbalanced</i>)	FCFS	118.338,47 $\pm$ 630,48	6.016,84 $\pm$ 14,43	88,61% $\pm$ 0,29%
	Prioridade	109.520,82 $\pm$ 612,23	6.016,84 $\pm$ 14,43	69,93% $\pm$ 0,93%
	Round-Robin ($q = 10$)	131.071,78 $\pm$ 650,77	19.103,97 $\pm$ 48,01	96,65% $\pm$ 0,04%
	TEJ (Proposto)	118.337,73 $\pm$ 630,41	6.016,84 $\pm$ 14,43	88,61% $\pm$ 0,29%

B. Análise das Trocas de Contexto

A Figura 2 evidencia a marcante diferença de custo operacional entre as políticas. Por serem políticas não preemptivas orientadas a término de rajada, FCFS, Prioridade e TEJ executaram rigorosamente a mesma quantidade de trocas de contexto nos cenários Equilibrado (6.016,84), CPU-Bound (1.999,49) e Desbalanceado (6.016,84). Essas trocas decorrem estritamente da alternância natural entre rajadas de CPU e bloqueios de E/S.

Em contrapartida, o Round-Robin gerou uma explosão de trocas de contexto: 19.103,97 no cenário Equilibrado (aumento de 217%) e 25.878,36 no cenário CPU-Bound (aumento de 1.194% em relação às políticas não preemptivas). No cenário I/O-Bound, como as rajadas de CPU são curtas ($[1, 5] \leq q = 10$), o Round-Robin raramente sofre preempção por tempo, resultando em trocas equivalentes (19.990,79) devidas unicamente aos frequentes bloqueios de E/S.

C. Análise de Justiça e Inanição (Jain do Slowdown)

A métrica de justiça de Jain (Figura 3) revela o principal mérito e a eficácia do algoritmo TEJ. O algoritmo de Prioridade Clássica apresentou os piores índices de justiça em todos os cenários (60,45% no Equilibrado; 62,08% no CPU-Bound; 69,93% no Desbalanceado). No cenário de Prioridades Desbalanceadas (onde 85% dos processos têm alta prioridade), processos comuns sofrem inanição severa na fila de prontos, elevando assimetricamente seus *slowdowns* individuais.

O algoritmo TEJ mitigou com sucesso esse problema: no cenário de Prioridades Desbalanceadas, o TEJ elevou o índice de Jain para **88,61% $\pm$ 0,29%**, empatando estatisticamente com o FCFS e reduzindo as distorções extremas de inanição. Embora o Round-Robin alcance o maior índice de Jain (96,65%), ele o faz às custas de um aumento de 217% a 1.194% no número de trocas de contexto e de uma severa degradação no turnaround médio.

VI. DISCUSSÃO CRÍTICA DIRECIONADA AO ALGORITMO TEJ

Em conformidade com as diretrizes da avaliação, a discussão a seguir articula as escolhas de projeto, métricas e *trade-offs* do algoritmo proposto (**TEJ**).

A. Em quais cenários o TEJ teve o melhor desempenho?

O TEJ obteve seu melhor desempenho relativo no **Cenário de Prioridades Desbalanceadas** (*unbalanced_priorities*). Neste ambiente estressante (85% dos processos com prioridade alta), o algoritmo de Prioridade Pura sofreu inanição severa, apresentando um Índice de Jain de apenas 69,93%. O TEJ elevou a justiça para **88,61% $\pm$ 0,29%**, reduzindo a espera indefinida dos processos de menor prioridade.

B. Em quais métricas o TEJ apresentou melhoria?

- **Justiça no Slowdown (Índice de Jain):** Aumento de +18,68 pontos percentuais no índice de Jain em relação à Prioridade Pura no cenário desbalanceado (69,93% $\rightarrow$ 88,61%) e +28,15 pontos no cenário equilibrado (60,45% $\rightarrow$ 88,60%).
- **Eficiência em Trocas de Contexto:** Por ser uma política não preemptiva, o TEJ manteve o número mínimo de trocas de contexto (6.016 no equilibrado e 1.999 no CPU-bound), superando a sobrecarga do Round-Robin (19.103 a 25.878 trocas).

C. Em quais cenários o TEJ teve desempenho inferior e quais custos introduziu?

No **Tempo Médio de Retorno (Turnaround)**, o TEJ apresentou desempenho inferior à Prioridade Pura (118.337,73 vs. 109.520,82 *ticks* no cenário desbalanceado).

Por que esse custo aconteceu? Trata-se do preço direto do mecanismo de resgate. Ao conceder precedência absoluta aos processos resgatados (fatos refletidos no *trade-off* de projeto da Tabela I), o TEJ interrompe a sequência estrita de despacho

das tarefas prioritárias mais curtas. Esse custo é o preço do mecanismo para reduzir a inanição sem comprometer as trocas de contexto.

D. Síntese Comparativa com o Round-Robin e Escolhas de Projeto

Embora o Round-Robin atinja um índice de Jain elevado (96,65%), ele o faz impondo a pior performance de turnaround global (196.865,69 *ticks* no CPU-bound) e uma sobrecarga de trocas de contexto até 1.194% maior. O TEJ apresenta um compromisso favorável entre justiça, turnaround e custo de troca de contexto: reduz a inanição, preserva a prioridade original e opera sem a sobrecarga de preempção.

E. Relação com a Literatura e Cenários de Vitória

O TEJ estabelece um meio-termo inovador em relação às abordagens clássicas da literatura:

- **Herança Conceitual:** Da mesma forma que as filas de Prioridade Estática e FCFS [1], o TEJ adota o modelo não preemptivo para suprimir custos de chaveamento. Em relação às técnicas tradicionais de envelhecimento (*aging*), ele herda a premissa de mitigar a inanição à medida que o tempo de espera avança, mas substitui as variações dinâmicas de prioridade por um limiar de resgate determinístico baseado em prazos rígidos.
- **Cenários de Vitória em Justiça sobre a Prioridade:** O TEJ supera o algoritmo de Prioridade Estática no índice de Jain nos cenários **Equilibrado** e de **Prioridades Desbalanceadas**. Nestes ambientes, a Prioridade Estática condena os processos de baixa prioridade à inanição (índices de Jain de 60,45% e 69,93%), enquanto o TEJ restabelece a equidade com índices estáveis de **88,60%** e **88,61%**, respectivamente.
- **Cenários de Vitória sobre o Round-Robin:** O TEJ vence o Round-Robin em **todos os cenários** no que tange à sobrecarga operacional (trocas de contexto) e tempo médio de retorno (*turnaround*). O Round-Robin apresenta uma degradação de até 1.194% em trocas de contexto e de 30% no tempo médio de retorno devido à preempção arbitrária, enquanto o TEJ opera com baixo custo de troca de contexto, equivalente às políticas não preemptivas avaliadas.

F. A Inexistência de Bala de Prata e Limitações do TEJ

Como amplamente consolidado na engenharia de software e na teoria de sistemas operacionais, não existe uma “bala de prata” (no silver bullet) no projeto de escalonamento [6]. A otimização de uma métrica frequentemente impõe a degradação de outra. As principais desvantagens e limitações identificadas no algoritmo TEJ são:

- **Penalização de Turnaround sob Alta Carga:** Ao resgatar processos de baixa prioridade que sofreriam inanição, o TEJ atrasa a execução de processos de alta prioridade. Isso gera um aumento no tempo médio de retorno global (de 109.520,82 para 118.337,73 *ticks* no cenário

desbalanceado), aproximando o turnaround do algoritmo ao do FCFS.

- **Latência de Resposta por Não Preempção:** Por ser não preemptivo, o TEJ não interrompe rajadas de CPU em andamento. Se uma tarefa de CPU-bound muito longa estiver executando, um novo processo altamente crítico ou que atingiu seu prazo de resgate precisará aguardar o término da rajada atual, o que o torna inadequado para sistemas de tempo real estrito ou interativos.
- **Parametrização Estática do Intervalo de Resgate:** O cálculo do prazo de resgate depende do parâmetro fixo `rescue_interval = 10`. Sob cargas dinâmicas flutuantes, um valor fixo pode ser ineficaz: se muito grande, ainda permite períodos temporários de inanição; se muito pequeno, descaracteriza as prioridades originais do sistema, fazendo o algoritmo convergir precocemente para o comportamento do FCFS.

VII. CONCLUSÃO

Este trabalho propôs o algoritmo Triagem com Espera Justa (TEJ) e avaliou seu desempenho comparativamente com três políticas tradicionais de escalonamento sob 1.600 simulações com rigor estatístico (IC 95%). O TEJ comprovou sua eficácia como solução determinística contra inanição (*starvation*) em cenários de alta assimetria de prioridades, elevando o índice de justiça de Jain de 69,93% para 88,61% sem incorrer na sobrecarga proibitiva de trocas de contexto do Round-Robin.

REFERENCES

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: John Wiley & Sons, 2018.
- [2] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA: Pearson, 2014.
- [3] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Boston, MA: Pearson, 2017.
- [4] D. P. Bovet and M. Cesati, *Understanding the Linux Kernel*, 3rd ed. Sebastopol, CA: O'Reilly Media, 2005.
- [5] R. Jain, D.-M. Chiu, and W. R. Hawe, “A quantitative measure of fairness and discrimination for resource allocation in shared computer systems,” Digital Equipment Corporation, Tech. Rep. DEC-TR-301, 1984.
- [6] F. P. Brooks, “No silver bullet: Essence and accidents of software engineering,” *Computer*, vol. 20, no. 4, pp. 10–19, 1987.